Atelier N°3 — Les formulaires HTML5

Matière	Atelier Développement Web 1 (ECUEF142)	Durée	3 h
Enseignante	Salma Daoud	Classe	TI1 — Semestre 1
Acquis visés	AA1 (structure et validation), AA4 (versionnement)	Projet fil rouge	Site Dar El Halwa — Sfax
Prérequis	TP2 validé : index.html et carte.html valides, dépôt à jour		

Objectifs de la séance

À l'issue de ce TP, l'étudiant est capable de :

- Structurer un formulaire avec form, fieldset et legend
- Choisir le type de champ adapté à chaque donnée
- Employer select, option, optgroup et textarea à bon escient.
- Contraindre la saisie avec les attributs natifs required, pattern, title, min, max, step, minlength et maxlength.
- Diagnostiquer un champ dont la contrainte ne s'applique pas, en identifiant l'attribut incompatible avec le type choisi.

Matériel et logiciels

- **VS Code** avec les extensions Live Server et HTML CSS Support
- **Git** (vérifier avec git --version)
- **Navigateur: exemple** Chrome ou Firefox
- Le dépôt dar-el-halwa, à jour depuis le TP2

Contexte : la pâtisserie souhaite recevoir des commandes de gâteaux personnalisés depuis son site. Vous allez créer la page qui recueille ces demandes. Aucune donnée ne sera réellement envoyée : le traitement serveur n'est pas au programme de ce module.

Travail demandé

Étape 1 — Créer la page commande.html

1. Ouvrir le projet dar-el-halwa dans VS Code et vérifier que git status affiche un arbre propre.
2. Dupliquer index.html sous le nom commande.html, puis modifier le titre de la page : Commander — Dar El Halwa.
3. Conserver l'en-tête, la navigation et le pied de page ; vider le contenu de main.
4. Ajouter le lien « `Commander` » dans le **nav** des trois pages.
5. Dans le main, écrire un titre de niveau 2 : `Commander un gâteau personnalisé`.

Point de contrôle 1 : les trois pages se répondent, et commande.html est déjà valide au W3C avant même le formulaire.

Étape 2 — Le conteneur du formulaire

1. Créer le formulaire. Il envoie ses données par la méthode `post` vers `traitement.php` — page qui n'existe pas encore, ce qui est normal.

```
<form action="traitement.php" method="post">    ... </form>
```

2. À l'intérieur, prévoir trois groupes de champs, chacun introduit par son intitulé : Vos coordonnées, Votre gâteau, Confirmation.

```
<fieldset>
    <legend>Vos coordonnées</legend>.........
</fieldset>
```

Question 1. Que se passe-t-il exactement quand on valide un formulaire dont l'attribut action pointe vers un fichier inexistant ? Testez à la fin du TP et décrivez ce que vous observez.

Question 2. Quelle différence y a-t-il entre method="post" et method="get" ? Dans quel cas les données apparaissent-elles dans l'URL ?

Étape 3 — Le groupe « Vos coordonnées »

1. Ajouter les champs suivants. Chaque champ possède un id, un name, et une étiquette qui lui est reliée.

Champ	Type	Contraintes
Nom	texte	obligatoire, 2 caractères minimum
Prénom	texte	obligatoire, 2 caractères minimum
Adresse électronique	courriel	obligatoire, exemple vous@exemple.tn
Téléphone	téléphone	obligatoire, 8 chiffres précédés de +216
Code postal	à déterminer	obligatoire, exactement 4 chiffres
Adresse de livraison	zone de texte	3 lignes, 40 colonnes, obligatoire
Gouvernorat	liste déroulante	Sfax, Tunis, Sousse, Gabès, Autre — Sfax par défaut

2. Le téléphone et le code postal se contraignent par une expression régulière dans l'attribut pattern, accompagnée d'un title expliquant le format attendu.

```
<p>
    <label for="telephone">Téléphone :</label>
    <input type="tel" id="telephone" name="telephone" pattern="\+216[0-9]{8}"
           title="+216 suivi de 8 chiffres" placeholder="+21622550211" required>
</p>
```

Attention : l'attribut for de l'étiquette reprend l'id du champ, jamais son name. C'est l'erreur la plus fréquente de ce TP.

Question 3. Ajoutez `maxlength="4"` à un champ de type `number`, puis essayez de saisir dix chiffres. Pourquoi l'attribut reste-t-il sans effet ? Quel type et quels attributs faut-il employer pour un code postal tunisien ?

> **Point de contrôle 2 :** cliquer sur le texte de chaque étiquette place le curseur dans le champ correspondant. Sept clics à tester.

Étape 4 — Le groupe « Votre gâteau »

1. Ajouter les champs suivants dans le deuxième groupe.

Champ	Type	Contraintes
Occasion	boutons radio	Anniversaire, Mariage, Fiançailles, Autre — un seul choix, obligatoire
Date de retrait	date	obligatoire, pas de date antérieure au 1er septembre 2026
Nombre de parts	nombre	de 6 à 100, par pas de 2, valeur initiale 12
Parfums souhaités	cases à cocher	Chocolat, Vanille, Pistache, Fruits rouges — Chocolat coché par défaut
Niveau de sucre	curseur	de 0 à 10, pas de 1, valeur initiale 5
Couleur du glaçage	couleur	valeur initiale #c8912a
Message sur le gâteau	texte	30 caractères maximum
Photo de modèle	fichier	images uniquement

2. Les boutons radio d'une même question partagent le même attribut name : c'est ce qui les rend mutuellement exclusifs.
3. Les cases à cocher d'un même groupe portent un name en tableau : parfums[].
4. Grouper les boutons radio dans un fieldset interne portant sa propre legend, et faire de même pour les cases à cocher.

```
<fieldset>
    <legend>Occasion</legend>
    <input type="radio" id="occ-anniv" name="occasion"  value="anniversaire" required>
    <label for="occ-anniv">Anniversaire</label>
    ...
</fieldset>
```

Question 4. Retirez temporairement l'attribut name commun aux quatre boutons radio et donnez-leur quatre noms différents. Que se passe-t-il à l'écran ? Qu'est-ce qui constitue le groupe, finalement ?

Question 5. L'attribut required n'est posé que sur un seul bouton radio du groupe. Le choix est-il tout de même obligatoire ? Vérifiez en envoyant le formulaire.

Étape 5 — La liste déroulante et la zone de texte

1. Revenir au champ Gouvernorat et vérifier que Sfax s'affiche par défaut grâce à l'attribut selected sur l'option correspondante.
2. Ajouter un champ « Langages de décoration » sous forme de liste à choix multiple, avec quatre options visibles.

```
<select multiple size="4" id="decors" name="decors[]">
    <option value="fleurs">Fleurs en pâte à sucre</option>
    <option value="fruits">Fruits frais</option>
    ...
</select>
```

3. Ajouter une zone de texte « Précisions » de 4 lignes et 40 colonnes, avec un maximum de 200 caractères.

Note : textarea n'a pas d'attribut value. Tout ce qui est écrit entre la balise ouvrante et la balise fermante devient le contenu du champ, espaces et retours à la ligne compris. Écrivez donc <textarea></textarea> sans rien entre les deux.

Question 6. Dans une liste déroulante, qu'est-ce qui est réellement envoyé au serveur : le texte affiché dans l'option, ou autre chose ? Justifiez à partir de votre code.

Étape 6 — Les boutons et la validation native

1. En fin de formulaire, hors des fieldset, ajouter un bouton d'envoi libellé « Envoyer ma commande » et un bouton de réinitialisation libellé « Tout effacer ».

```
<p>
    <button type="submit">Envoyer ma commande</button>
    <button type="reset">Tout effacer</button>
</p>
```

2. Tester successivement les cas du tableau ci-dessous et relever le message affiché par le navigateur.

Test	Résultat attendu
Envoyer le formulaire vide	Bulle « Veuillez renseigner ce champ » sur le premier champ fautif
Saisir abc dans Téléphone	Bulle affichant le texte de l'attribut title
Saisir 7 dans Nombre de parts	Message indiquant les deux valeurs valides les plus proches
Choisir une date au 15/08/2026	Message indiquant la date minimale autorisée

3. Changer la langue du navigateur et renvoyer le formulaire vide : le message change de langue sans que vous ayez écrit une seule ligne de code.

Question 7. Ajoutez l'attribut formnovalidate à un second bouton d'envoi, puis renvoyez le formulaire vide. Que se passe-t-il ? Dans quel cas réel cet attribut est-il utile ?

Point de contrôle 3 : les quatre tests du tableau produisent bien le message attendu, et le bouton « Tout efface » remet tous les champs à leur valeur de départ.

Étape 7 — Accessibilité du formulaire

1. Vérifier point par point :

• cliquer sur le texte de chaque étiquette place le curseur dans le champ ;

• le formulaire est entièrement parcourable au clavier avec la touche Tabulation, dans un ordre logique ;

• chaque fieldset possède une legend ;

• placeholder n'est jamais employé à la place d'une étiquette, seulement en complément.

Question 8. Un développeur supprime toutes les étiquettes et place leur texte dans les placeholder : « c'est plus joli et ça prend moins de place ». Donnez deux problèmes concrets que cela crée pour l'utilisateur.

Question 9. Ouvrez l'inspecteur, supprimez l'attribut required d'un champ obligatoire, puis renvoyez le formulaire. Que concluez-vous sur la nature de cette validation ?

Étape 8 — Valider et publier

1. Valider commande.html au W3C : zéro erreur.
2. Consigner dans la fiche de correctifs du README.md au moins deux problèmes rencontrés pendant la séance : symptôme, cause identifiée, correction appliquée.
3. Committer au fur et à mesure des unités de travail, puis pousser :

```
git add commande.html index.html carte.html
```

```
git commit -m "Creer la page de commande personnalisee"
```

```
git push
```

Point de contrôle 4 : `git log --oneline` affiche l'historique des trois séances, et le dépôt distant reflète l'état local.

Extension bonus

1. Ajouter un champ « Site web ou page Instagram » de type url, non obligatoire, et un champ de type time pour l'heure de retrait souhaitée.

2. Documenter dans le README.md la liste complète des types HTML5 employés dans le projet, avec pour chacun le champ concerné.

3. Regrouper les options de la liste des décors sous deux intitulés à l'aide de optgroup : « Décors sucrés » et « Décors frais ».